CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 25: Restricted Boltzmann Machine Energy and Reconstruction

Problem Statement: A movie-streaming service wants to learn hidden taste patterns from users' binary watched/not-watched records and fill in likely preferences. A binary user–movie matrix (or binarised MNIST digits) with 784 visible units and 64 hidden units. The objective is to implement an RBM, compute its energy function and reconstruct inputs. Use NumPy, SciPy, Matplotlib.

Tasks:

1. Implement the energy function $E(v, h) = -aTv - bTh - vTWh$ in NumPy.

2. Compute hidden activation probabilities p(h|v) and visible probabilities p(v|h).

3. Train the RBM with one-step contrastive divergence.

4. Plot average free energy and reconstruction error over epochs.

5. Show original and reconstructed samples side by side.

FORMULA:

1. RBM Energy Function

$E(v,h) = -a.T \times v - b.T \times h - v.T \times W \times h$

2. Hidden Layer Activation Probability

$P(h=1|v) = 1 / (1 + \exp(-(b + W.T \times v)))$

3. Visible Layer Activation Probability

$P(v=1|h) = 1 / (1 + \exp(-(a + W \times h)))$

4. Sigmoid Activation Function

$Sigmoid(x) = 1 / (1 + \exp(-x))$

5. Contrastive Divergence Weight Update

$W(new) = W(old) + Learning\ Rate \times (v0 \times h0.T - vk \times hk.T)$

6. Visible Bias Update

$a(new) = a(old) + Learning\ Rate \times (v0 - vk)$

7. Hidden Bias Update

$b(new) = b(old) + Learning\ Rate \times (h0 - hk)$

8. Free Energy

$F(v) = -a.T \times v - Sum(\log(1 + \exp(b + W.T \times v)))$

9. Reconstruction Error

$MSE = Sum((Original - Reconstructed)^2) / Number\ of\ Elements$

10. Average Free Energy

$Average\ Free\ Energy = Sum(Free\ Energy) / Number\ of\ Samples$

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.special import expit

from sklearn.datasets import load_digits
```

```
# Load dataset

digits = load_digits()

# Resize images to 28 x 28

from skimage.transform import resize

X = np.array([

    resize(img, (28, 28), anti_aliasing=True)

    for img in digits.images

])

# Normalize and binarize data

X = X.reshape(-1, 784)

X = (X > 0.2).astype(float)

# Initialize RBM

np.random.seed(42)

n_visible = 784

n_hidden = 64

learning_rate = 0.01

epochs = 30

batch_size = 32


W = np.random.normal(0, 0.01,
```

```
                (n_visible, n_hidden))
a = np.zeros(n_visible)
b = np.zeros(n_hidden)
free_energy = []
reconstruction_error = []
# Energy function
def energy(v, h):
    return -np.dot(v, a) - np.dot(h, b) - np.dot(v, W @ h)
# Hidden probability
def hidden_prob(v):
    return expit(b + np.dot(v, W))
# Visible probability
def visible_prob(h):
    return expit(a + np.dot(h, W.T))
# Free energy
def free_energy_value(v):
    return -np.dot(v, a) - np.sum(
        np.logaddexp(0, b + np.dot(v, W))
    )
# Training using CD-1
```

```
for epoch in range(epochs):
    np.random.shuffle(X)
    total_error = 0
   total_energy = 0
    for i in range(0, len(X), batch_size):
        v0 = X[i:i + batch_size]
        # Positive phase
        ph0 = hidden_prob(v0)
        h0 = (np.random.rand(*ph0.shape) < ph0).astype(float)
        # Negative phase
        pv1 = visible_prob(h0)
        v1 = (np.random.rand(*pv1.shape) < pv1).astype(float)
        ph1 = hidden_prob(v1)
        # Update weights and biases
        W += learning_rate * (
            np.dot(v0.T, ph0) -
            np.dot(v1.T, ph1)
        ) / len(v0)
        a += learning_rate * np.mean(v0 - v1, axis=0)
        b += learning_rate * np.mean(ph0 - ph1, axis=0)
```

```
        # Calculate error
        total_error += np.mean((v0 - pv1) ** 2)
        total_energy += np.mean(
            [free_energy_value(v) for v in v0]
        )
    reconstruction_error.append(
        total_error / int(np.ceil(len(X) / batch_size))
    )
    free_energy.append(
        total_energy / int(np.ceil(len(X) / batch_size))
    )

    print("Epoch:", epoch + 1,
          "Error:", round(reconstruction_error[-1], 4))
# Plot free energy
plt.figure(figsize=(7, 4))
plt.plot(free_energy)
plt.title("Average Free Energy")
plt.xlabel("Epochs")
plt.ylabel("Free Energy")
```

```
plt.grid()

plt.show()

# Plot reconstruction error

plt.figure(figsize=(7, 4))

plt.plot(reconstruction_error)

plt.title("Reconstruction Error")

plt.xlabel("Epochs")

plt.ylabel("Error")

plt.grid()

plt.show()


# Display original and reconstructed images

sample = X[:5]

hidden = hidden_prob(sample)

reconstructed = visible_prob(hidden)

plt.figure(figsize=(10, 4))

for i in range(5):

    plt.subplot(2, 5, i + 1)

    plt.imshow(sample[i].reshape(28, 28), cmap="gray")

    plt.title("Original")
```

```
    plt.axis("off")
    plt.subplot(2, 5, i + 6)
    plt.imshow(reconstructed[i].reshape(28, 28), cmap="gray")
    plt.title("Reconstructed")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT:

```
Epoch: 1 Error: 0.1352
Epoch: 2 Error: 0.1254
Epoch: 3 Error: 0.125
Epoch: 4 Error: 0.1253
Epoch: 5 Error: 0.1254
Epoch: 6 Error: 0.1253
Epoch: 7 Error: 0.1251
Epoch: 8 Error: 0.1253
Epoch: 9 Error: 0.1253
Epoch: 10 Error: 0.1253
Epoch: 11 Error: 0.1253
Epoch: 12 Error: 0.1254
```

Epoch: 13 Error: 0.1253

Epoch: 14 Error: 0.1253

Epoch: 15 Error: 0.125

Epoch: 16 Error: 0.1257

Epoch: 17 Error: 0.1251

Epoch: 18 Error: 0.1253

Epoch: 19 Error: 0.1251

Epoch: 20 Error: 0.1253

Epoch: 21 Error: 0.1254

Epoch: 22 Error: 0.1252

Epoch: 23 Error: 0.1252

Epoch: 24 Error: 0.1252

Epoch: 25 Error: 0.1252

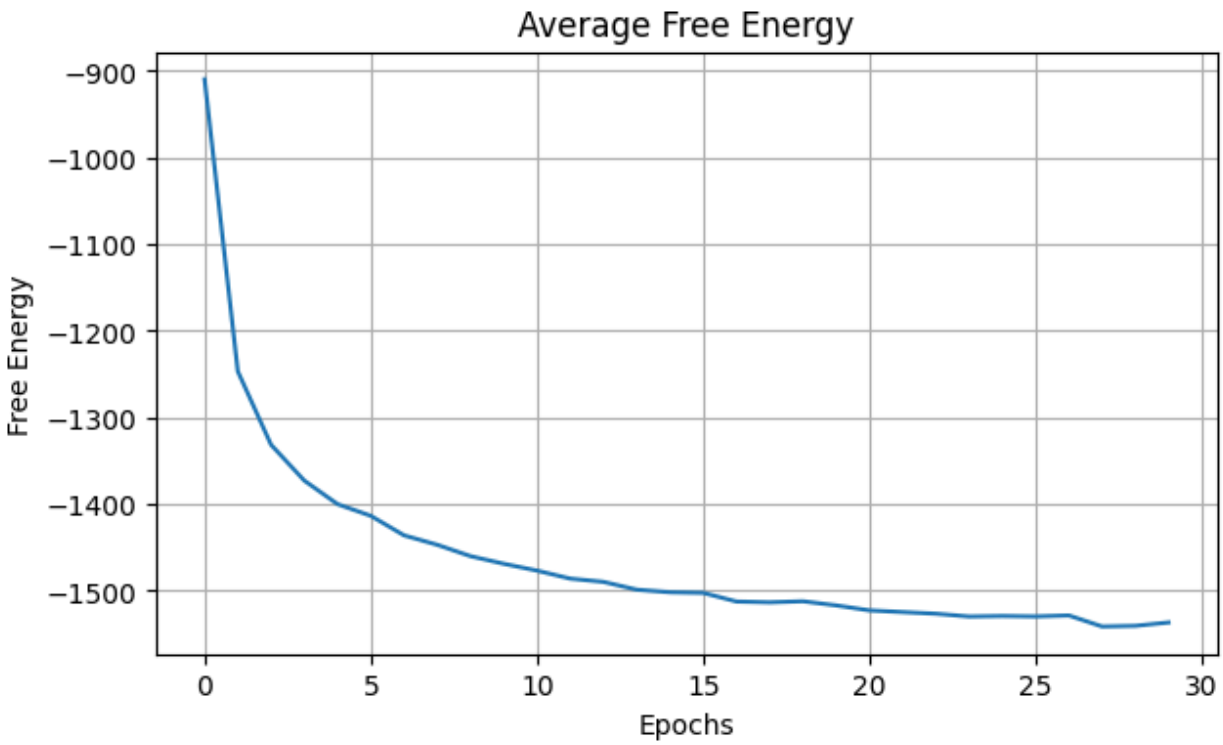
Epoch: 26 Error: 0.1254

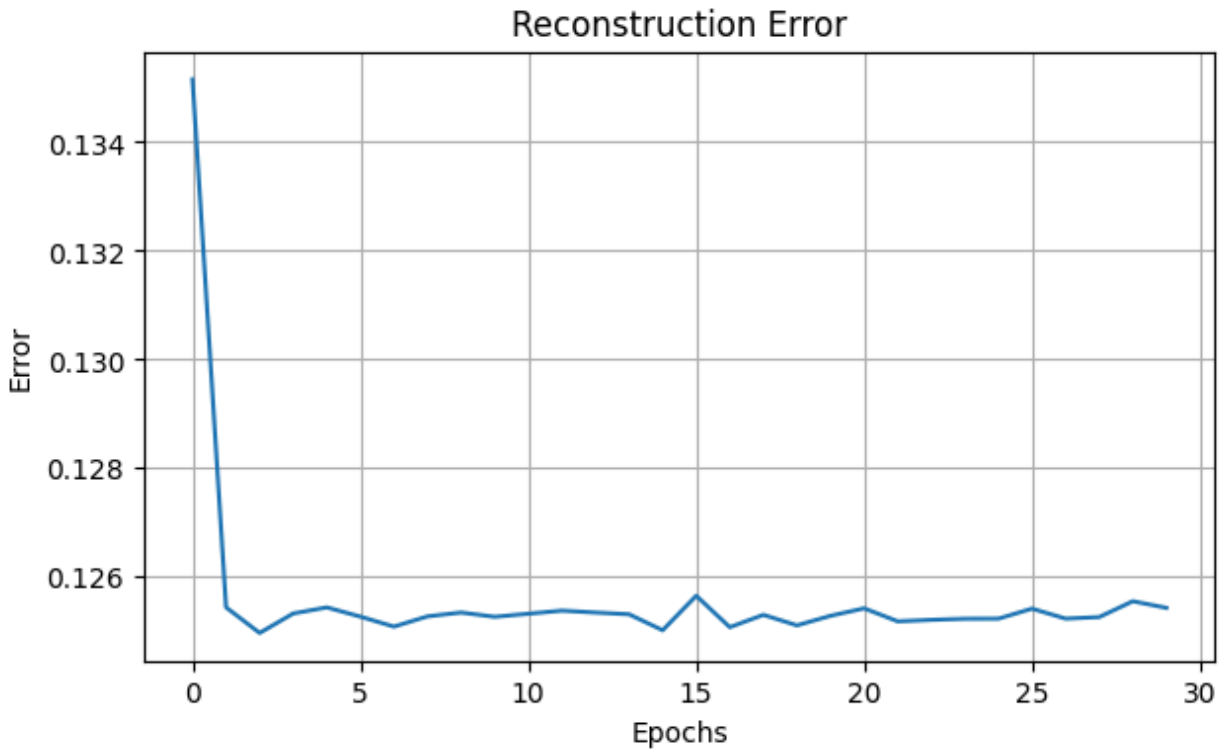
Epoch: 27 Error: 0.1252

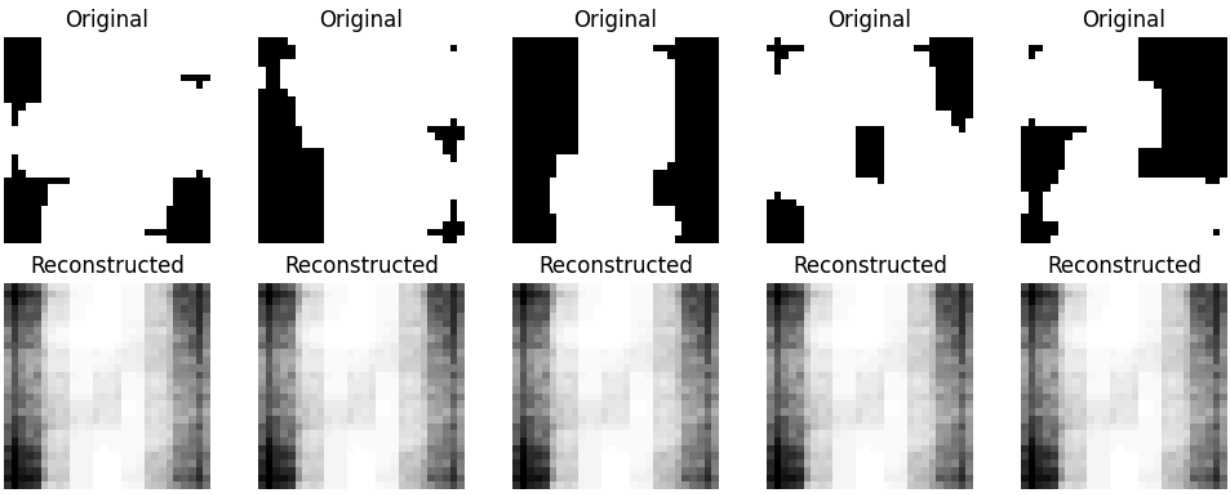
Epoch: 28 Error: 0.1253

Epoch: 29 Error: 0.1256

Epoch: 30 Error: 0.1254


Average Free Energy
Free Energy
−900
−1000
−1100
−1200
−1300
−1400
−1500
0
5
10
15
20
25
30
Epochs


Reconstruction Error
Error
0.134
0.132
0.130
0.128
0.126
0
5
10
15
20
25
30
Epochs


Original
Original
Original
Original
Original
Reconstructed
Reconstructed
Reconstructed
Reconstructed
Reconstructed